Nonlinear Programming Study Set

Question 1. *If K units of capital and L units of labor are used, a company can produce KL units of a manufactured good. Capital can be purchased at \$4/unit and labor can be purchased at \$1/unit. A total of \$8 is available to purchase capital and labor. How can the firm maximize the quantity of the good that can be manufactured?*

CorrectAnswer 1. *Let K be the units of capital purchased and L be the units of labor purchased. Then K and L must satisfy $4K + L \leq 8$, $K \geq 0$, and $L \geq 0$. Thus, the firm wants to solve the following constrained maximization problem:*

$$\begin{aligned} \textit{maximize} \quad & z = KL \\ \textit{subject to} \quad & 4K + L \leq 8 \\ & K, L \geq 0 \end{aligned}$$

```python
import numpy as np
from scipy.optimize import minimize

# Define the objective function
def objective(x):
    K, L = x
    return -K * L  # Negative because we are minimizing

# Define the constraint
def constraint(x):
    K, L = x
    return 8 - (4 * K + L)

# Define bounds for K and L
bounds = [(0, None), (0, None)]  # Both K and L are greater than or equal
    to 0

# Define initial guess
x0 = [1, 1]  # Initial guess for K and L

# Define the constraints in the format required by scipy
constraints = [{'type': 'ineq', 'fun': constraint}]

# Run the optimization
result = minimize(objective, x0, method='SLSQP', bounds=bounds, constraints
    =constraints)

# Print the results
if result.success:
    optimized_K, optimized_L = result.x
    print(f"Optimal value of K: {optimized_K}")
    print(f"Optimal value of L: {optimized_L}")
    print(f"Maximum value of z: {-result.fun}")  # Negate again to get the
        maximum
else:
    print("Optimization failed:", result.message)
```

Nonlinear Programming Study Set

Question 2. *Oily produces three types of gasoline: regular, unleaded, and premium. All three are produced by combining lead and crude oil brought in from Alaska and Texas. The required sulphur content, octane levels, minimum daily demand (in gallons), and sales price per gallon of each type of gasoline are given in Table below. The crude brought in from Alaska is made by blending two types of crude: Alaska1 and Alaska2. The Alaska crude is blended in Alaska and shipped via pipeline to Oily's Texas refinery. At most, 10,000 gallons of crude per day can be shipped from Alaska. The sulphur content, octane level, daily maximum amount available (in gallons) and purchase cost (per gallon) for each type of Alaska crude, Texas crude, and lead are given in Table . Of course, unleaded gasoline can contain no lead. Formulate an NLP to help Oily maximize the daily profit obtained from selling gasoline.*

Type of Gasoline	Sulphur Content (%)	Octane Level	Minimum Daily Demand (Gallons)	Sales Price
Regular	≤ 3	≥ 90	5,000	0.86
Unleaded	≤ 3	≥ 88	5,000	0.93
Premium	≤ 2.8	≥ 94	5,000	1.06

Type of Input	Sulphur Content (%)	Octane Level	Maximum Availability (Gallons)	Cost (per gallon)
Alaska 1	4	91	0	0.78
Alaska 2	1	97	0	0.88
Texas	2	83	11,000	0.75
Lead	0	800	6,000	1.30

CorrectAnswer 2.

We define the following decision variables:

- R = *gallons of regular gasoline produced daily*
- U = *gallons of unleaded gasoline produced daily*
- P = *gallons of premium gasoline produced daily*
- $A1$ = *gallons of Alaska1 crude purchased daily*
- $A2$ = *gallons of Alaska2 crude purchased daily*
- T = *gallons of Texas crude purchased daily*
- L = *gallons of lead purchased daily*
- SA = *sulphur content of crude purchased from Alaska*
- OA = *octane level of crude purchased from Alaska*
- A = *total gallons of crude purchased from Alaska*
- LP = *gallons of lead used daily to make premium gasoline*
- TP = *gallons of Texas crude used daily to make premium gasoline*
- AP = *gallons of Alaska crude used daily to make premium gasoline*
- TU = *Texas crude used daily to make unleaded gasoline*
- AU = *Alaska crude used daily to make unleaded gasoline*
- AR = *Alaska crude used daily to make regular gasoline*
- TR = *Texas crude used daily to make regular gasoline*
- LR = *gallons of lead used daily to make regular gasoline*

The objective function maximizes daily revenues (86 * R + 93 * U + 106 * P) less the daily costs of purchasing crude (78 * A1 + 88 * A2 + 75 * T + 130 * L). So the model becomes:

$$\text{Maximize: } 86 \cdot R + 93 \cdot U + 106 \cdot P - 78 \cdot A1 - 88 \cdot A2 - 75 \cdot T - 130 \cdot L \quad (1)$$

Subject to:

$$A < 10000 \quad (2)$$
$$T < 10000 \quad (3)$$
$$L < 11000 \quad (4)$$
$$R < 6000 \quad (5)$$
$$U > 5000 \quad (6)$$
$$P > 5000 \quad (7)$$
$$SA = (0.04 \cdot A1 + 0.01 \cdot A2) / A \quad (8)$$
$$OA = (91 \cdot A1 + 97 \cdot A2) / A \quad (9)$$
$$A = A1 + A2 \quad (10)$$
$$P = LP + TP + AP \quad (11)$$
$$U = TU + AU \quad (12)$$
$$L = LP + LR \quad (13)$$
$$A = AP + AU + AR \quad (14)$$
$$T = TP + TU + TR \quad (15)$$
$$(AR \cdot OA + 83 \cdot TR + 800 \cdot LR) / R > 90 \quad (16)$$
$$(AP \cdot OA + 83 \cdot TP + 800 \cdot LP) / P > 94 \quad (17)$$
$$(AU \cdot OA + TU \cdot 83) / U > 88 \quad (18)$$
$$SA \cdot AR + 0.02 \cdot TR/R < 0.03 \quad (19)$$
$$SA \cdot AP + 0.02 \cdot TP/P < 0.028 \quad (20)$$
$$SA \cdot AU + 0.02 \cdot TU/U < 0.03 \quad (21)$$
$$LP, TP, AP, TU, AU, LR, TR, AR > 0 \quad (22)$$
$$R = TR + AR + LR \quad (23)$$

```
from gurobipy import Model, GRB

# Create a new model
m = Model("optimization")

# Create variables
R = m.addVar(vtype=GRB.CONTINUOUS, name="R")
U = m.addVar(vtype=GRB.CONTINUOUS, name="U")
P = m.addVar(vtype=GRB.CONTINUOUS, name="P")
A1 = m.addVar(vtype=GRB.CONTINUOUS, name="A1")
A2 = m.addVar(vtype=GRB.CONTINUOUS, name="A2")
T = m.addVar(vtype=GRB.CONTINUOUS, name="T")
L = m.addVar(vtype=GRB.CONTINUOUS, name="L")
LP = m.addVar(vtype=GRB.CONTINUOUS, name="LP")
```

```
TP = m.addVar(vtype=GRB.CONTINUOUS, name="TP")
AP = m.addVar(vtype=GRB.CONTINUOUS, name="AP")
TU = m.addVar(vtype=GRB.CONTINUOUS, name="TU")
AU = m.addVar(vtype=GRB.CONTINUOUS, name="AU")
LR = m.addVar(vtype=GRB.CONTINUOUS, name="LR")
TR = m.addVar(vtype=GRB.CONTINUOUS, name="TR")
AR = m.addVar(vtype=GRB.CONTINUOUS, name="AR")

# Set the objective
m.setObjective(86 * R + 93 * U + 106 * P - 78 * A1 - 88 * A2 - 75 * T - 130
    * L, GRB.MAXIMIZE)

# Add linear constraints
m.addConstr(A1 + A2 <= 10000, "Constraint_A")
m.addConstr(T <= 10000, "Constraint_T")
m.addConstr(L <= 11000, "Constraint_L")
m.addConstr(R <= 6000, "Constraint_R")
m.addConstr(U >= 5000, "Constraint_U")
m.addConstr(P >= 5000, "Constraint_P")

m.addConstr(AP + AU + AR == A1 + A2, "Constraint_A_Sum")
m.addConstr(LP + TP + AP == P, "Constraint_P_Sum")
m.addConstr(TU + AU == U, "Constraint_U_Sum")
m.addConstr(LP + LR == L, "Constraint_L_Sum")
m.addConstr(TP + TU + TR == T, "Constraint_T_Sum")
m.addConstr(TR + AR + LR == R, "Constraint_R_Sum")

# Add quadratic constraints
m.addQConstr((AR * (91 * A1 + 97 * A2) + 83 * TR + 800 * LR) >= 90*R, "
    Quadratic_Constraint_1")
m.addQConstr((AP * (91 * A1 + 97 * A2) + 83 * TP + 800 * LP) >= 94*P, "
    Quadratic_Constraint_2")
m.addQConstr((AU * (91 * A1 + 97 * A2) + TU * 83) >= 88*U, "
    Quadratic_Constraint_3")

m.addQConstr((0.04 * A1 + 0.01 * A2) * AR + 0.02 * TR <= R* 0.03 * (A1 + A2
    ), "Quadratic_Constraint_4")
m.addQConstr((0.04 * A1 + 0.01 * A2) * AP + 0.02 * TP <= P* 0.028 * (A1 +
    A2), "Quadratic_Constraint_5")
m.addQConstr((0.04 * A1 + 0.01 * A2) * AU + 0.02 * TU <= U* 0.03 * (A1 + A2
    ), "Quadratic_Constraint_6")

m.setParam('NonConvex', 2)

# Optimize the model
m.optimize()

# Print the solution
for v in m.getVars():
    print(f'{v.varName}: {v.x}')
print(f'Obj: {m.objVal}')
```

Question 3. *Show that for* $S = R^3$, $f(x_1, x_2, x_3) = x_1^2 + x_2^2 + 3x_3^2 - x_1x_2 - x_2x_3 - x_1x_3$ *is a convex function.*

CorrectAnswer 3. *The Hessian is given by*

$$H(x_1, x_2, x_3) = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 4 \end{bmatrix}$$

We compute the eigenvalues: $[0.44, 3.00, 4.56]$, *all of them are positive. Therefore,* $f(x_1, x_2, x_3)$ *is convex.*

```
m = np.matrix([[2, -1, -1],[-1,2,-1],[-1,-1,4]])
print(np.linalg.eigvals(m))
```

Question 4. *A monopolist producing a single product has two types of customers. If q_1 units are produced for customer 1, then customer 1 is willing to pay a price of $70 - 4q_1$ dollars. If q_2 units are produced for customer 2, then customer 2 is willing to pay a price of $150 - 15q_2$ dollars. For $q > 0$, the cost of manufacturing q units is $100 + 15q$ dollars. To maximize profit, how much should the monopolist sell to each customer?*

CorrectAnswer 4.
Let $f(q_1, q_2)$ be the monopolist's profit if she produces q_i units for customer i. Then (assuming some production takes place):

$$f(q_1, q_2) = q_1(70 - 4q_1) + q_2(150 - 15q_2) - 100 - 15q_1 - 15q_2$$

We will first look at the Hessian matrix to understand the curvature. The Hessian is given by

$$H(q_1, q_2) = \begin{bmatrix} -8 & 0 \\ 0 & -30 \end{bmatrix}$$

The eigen values are both negative; therefore, the function is concave. We can solve this problem using Gurobi as this is a concave function and we are looking for the global maximum.

```
matrix = np.matrix([[-8,0],[0,-30]])
print(np.linalg.eigvals(matrix))

# Create a new model
m = Model("concave_optimization")

# Create variables
q1 = m.addVar(vtype=GRB.CONTINUOUS, name="q1")
q2 = m.addVar(vtype=GRB.CONTINUOUS, name="q2")

# Set the objective function
obj = q1 * (70 - 4 * q1) + q2 * (150 - 15 * q2) - 100 - 15 * q1 - 15 * q2
m.setObjective(obj, GRB.MAXIMIZE)

# Since the objective is a concave function, we need to set the NonConvex
    parameter
m.setParam('NonConvex', 2)

# Optimize the model
m.optimize()

# Print the solution
if m.status == GRB.OPTIMAL:
    print(f'Optimal solution found: q1 = {q1.x}, q2 = {q2.x}')
    print(f'Maximum value of the objective function: {m.objVal}')
else:
    print("No optimal solution found")
```

Question 5. *Use the method of steepest descent to approximate the solution to*

$$maximize \quad Z = (x_1 - 3)^2 + (x_2 - 2)^2 = f(x_1, x_2)$$

$$s.t. (x_1, x_2) \in R^2$$

Begin at the point $x_0 = (1, 1)$ *and use your step size as* 0.01.

CorrectAnswer 5. *We find the optimal solution to be* $x = (3, 2)$ *and this took us 59 iterations!*

```
import numpy as np  # Support for large, multi-dimensional arrays and
    matrices
import numpy.linalg as la  # General linear algebra package

# Defines a function f(x), which takes an input array x and returns a
    scalar value.
def f(x):
    # Define the objective function
    return (x[0]-3) ** 2 + (x[1]-2)**2   # python starts indexing from 0!

#Defines the gradient df(x) of f(x). It returns the derivative of f with
    respect to x.
def df(x):
    # Define the gradient of the objective function
    return np.array([2 * (x[0]-3), 2*(x[1]-2)])

#Gradient Descent Initialization
points = [np.array([1, 1])]
tol = 1e-6
maxiter = 1000
dxmin = 1e-6
dx = float('Inf')
gnorm = float('Inf')

count = 0
t_opt = 0.1

while (gnorm >= tol and (count <= maxiter and dx >= dxmin)):
    x = points[-1]
    g = -1 * df(x)
    next_point = x + t_opt * g
    points.append(next_point)

    # Update termination conditions
    gnorm = la.norm(-g)
    dx = la.norm(next_point - x)
    count = count + 1

    # Output information for user
    print("The current point is: ", x)
    print("g = ", g)

# Final output to user
print("Numbers of iterations: ", count - 1)
```

Question 6. *Use the method of steepest descent to approximate the solution to*

$$\textit{minimize} \quad Z = (x_1 - 1)^2 + (x_2 - 1)^2 + |x_1 + x_2 - 3| = f(x_1, x_2)$$

$$s.t. (x_1, x_2) \in R^2$$

Begin at the point $x_0 = (0, 0)$ *and use your step size as* 0.01*, this time decrease your termination tolerance to* 0.001*.*

CorrectAnswer 6. *The gradient, considering the piecewise definition due to the absolute value:*

$$\nabla Z = \begin{cases} \left[2(x_1 - 1) + 1, 2(x_2 - 1) + 1\right] & \text{if } x_1 + x_2 - 3 > 0 \\ \left[2(x_1 - 1) - 1, 2(x_2 - 1) - 1\right] & \text{if } x_1 + x_2 - 3 < 0 \\ \left[2(x_1 - 1) + \alpha, 2(x_2 - 1) + \alpha\right] \text{ where } -1 \le \alpha \le 1 & \text{if } x_1 + x_2 - 3 = 0 \end{cases}$$

```
# Define the objective function
def f(x):
    return (x[0] - 1)**2 + (x[1] - 1)**2 + np.abs(x[0] + x[1] - 3)

# Define the gradient of the objective function
def df(x):
    # Compute the gradient for the quadratic parts
    grad = np.array([2 * (x[0] - 1), 2 * (x[1] - 1)])
    # Handle the absolute value term
    if x[0] + x[1] - 3 > 0:
        grad += np.array([1, 1])
    elif x[0] + x[1] - 3 < 0:
        grad += np.array([-1, -1])
    return grad

# Gradient Descent Initialization
points = [np.array([0, 0])]
tol = 1e-3
maxiter = 1000
dxmin = 1e-3
dx = float('Inf')
gnorm = float('Inf')

count = 0
t_opt = 0.1  # Step size
```

```
while gnorm >= tol and count <= maxiter and dx >= dxmin:
    x = points[-1]
    g = -df(x)
    next_point = x + t_opt * g
    points.append(next_point)

    # Update termination conditions
    gnorm = la.norm(g)
    dx = la.norm(next_point - x)
    count += 1

    # Output information for user
    print("Iteration:", count)
    print("The current point is:", x)
    print("Gradient =", g)
    print("Function value =", f(x))
    print("----")

# Final output to user
print("Final point:", points[-1])
print("Final function value:", f(points[-1])
```

Question 7. *A company is planning to spend \$10,000 on advertising. It costs \$3,000 per minute to advertise on television and \$1,000 per minute to advertise on radio. If the firm buys x minutes of television advertising and y minutes of radio advertising, then its revenue in thousands of dollars is given by* $f(x,y) = -2x^2 - y^2 + xy + 8x + 3y$. *How can the firm maximize its revenue?*

CorrectAnswer 7.
This problem can be solved analytically by the help of Lagrange multipliers, as shown below.

$$\begin{aligned} \max z = & -2x^2 - y^2 + xy + 8x + 3y \\ s.t. \quad & 3x + y = 10 \end{aligned}$$

Then the Lagrangian $L(x,y,\lambda)$ *is:* $L(x,y,\lambda) = -2x^2 - y^2 + xy + 8x + 3y + \lambda(10 - 3x - y)$. *We set*

$$\frac{\partial L}{\partial x} = -4x + y + 8 - 3\lambda = 0 \qquad (1)$$

$$\frac{\partial L}{\partial y} = -2y + x + 3 - \lambda = 0 \qquad (2)$$

$$\frac{\partial L}{\partial \lambda} = 10 - 3x - y = 0 \qquad (3)$$

This yields

$$\begin{aligned} &\text{From } (1): y = 3\lambda - 8 + 4x, \\ &\text{From } (2): x = \lambda - 3 + 2y. \\ &\text{Thus, } y = 3\lambda - 8 + 4(\lambda - 3 + 2y) = 7\lambda - 20 + 8y, \\ &\text{or } y = \frac{20}{7} - \lambda, \\ &x = \lambda - 3 + 2\left(\frac{20}{7} - \lambda\right) = \frac{19}{7} - \lambda. \qquad (4)(5) \end{aligned}$$

Substituting (4) and (5) into (3) yields

$$10 - 3\left(\frac{19}{7} - \frac{\lambda}{7}\right) - \left(\frac{20}{7} - \frac{\lambda}{7}\right) = 0, \text{ or } 4\lambda - 1 = 0,$$

$$\text{or } \lambda = \frac{1}{4}. \qquad (25)$$

Then (23) and (24) yield

$$\begin{aligned} \bar{y} &= \frac{20}{7} - \frac{1}{4} = \frac{73}{28}, \\ \bar{x} &= \frac{19}{7} - \frac{1}{4} = \frac{69}{28}. \end{aligned} \qquad (26)$$

The Hessian for $f(x,y)$ *is*

$$H(x,y) = \begin{bmatrix} -4 & 1 \\ 1 & -2 \end{bmatrix}.$$

The constraint is linear and $f(x,y)$ *is concave; so, the Lagrange multiplier method does yield the optimal solution to the NLP.*

```python
from scipy.optimize import minimize

# Objective function (we want to maximize z, so we minimize -z)
def objective(x):
    return 2*x[0]**2 + x[1]**2 - x[0]*x[1] - 8*x[0] - 3*x[1]

# Constraint equations
def constraint(x):
    return 3*x[0] + x[1] - 10

# Initial guess
x0 = [0, 0]

# Define the constraint as a dictionary
con = {'type': 'eq', 'fun': constraint}

# Solve the problem using SLSQP method
sol = minimize(objective, x0, constraints=con, method='SLSQP')

# Print the results
print(f"The solution is at x = {sol.x[0]} and y = {sol.x[1]}")
print(f"The maximum value of the objective function is {-sol.fun}")
print(f"----------------------------------------------------------------------")
print(f"The Lagrangian solution is x = 2.46428 and y = 2.6071")
```

Nonlinear Programming Study Set

Question 8. *Chemical Brothers must determine how many barrels of chemical to extract during each of the next two years. If Chemical Brothers extracts x_1 million barrels during year 1, each barrel can be sold for $\$30 - x_1$. If Chemical Brothers extracts x_2 million barrels during year 2, each barrel can be sold for $\$35 - x_2$. The cost of extracting x_1 million barrels during year 1 is x_2 1 million dollars, and the cost of extracting x_2 million barrels during year 2 is $2x_2^2$ million dollars. A total of 20 million barrels of chemical are available, and at most \$250 million can be spent on extraction. Formulate an NLP to help Chemical Brothers maximize profits (revenues less costs) for the next two years.*

CorrectAnswer 8. *Define*

$$x_1 = \textit{millions of barrels of chemical extracted during year 1}$$
$$x_2 = \textit{millions of barrels of chemical extracted during year 2}$$

Then the appropriate NLP is

$$\begin{aligned} \max z &= x_1(30 - x_1) + x_2(35 - x_2) - x_1^2 - 2x_2^2 \\ &= 30x_1 + 35x_2 - 2x_1^2 - 3x_2^2 \\ s.t. \quad & x_1^2 + 2x_2^2 \leq 250 \\ & x_1 + x_2 \leq 20 \\ & x_1, x_2 \geq 0 \end{aligned}$$

```
# Define the objective function
def objective(x):
    # Since it's a maximization problem, we need to negate the function
    return -(30*x[0] + 35*x[1] - 2*x[0]**2 - 3*x[1]**2)

# Define the constraints
def constraint1(x):
    return 250 - (x[0]**2 + 2*x[1]**2)

def constraint2(x):
    return 20 - (x[0] + x[1])

# Initial guesses
n = 2
x0 = np.zeros(n)

# Setup the bounds for x1 and x2 (both are non-negative)
b = (0, None)
bnds = (b, b)

# Define the constraints in dictionary format
con1 = {'type': 'ineq', 'fun': constraint1}
con2 = {'type': 'ineq', 'fun': constraint2}
cons = [con1, con2]

# Run the optimization
sol = minimize(objective, x0, method='SLSQP', bounds=bnds, constraints=cons
    )

# Optimal values
x1\_optimal, x2\_optimal = sol.x
objective_value = -sol.fun

print("Optimal x1:", x1_optimal)
print("Optimal x2:", x2_optimal)
print("Maximum value of the objective function:", objective_value)
```

Nonlinear Programming Study Set

Question 9. *I have \$1,000 to invest in three stocks. Let Si be the random variable representing the annual return on \$1 invested in stock i. Thus, if $S_i = 0.12$, \$1 invested in stock i at the beginning of a year was worth \$1.12 at the end of the year. We are given the following information: $E(S_1) = 0.14$, $E(S_2) = 0.11$, $E(S_3) = 0.10$, $var(S_1) = 0.20$, $var(S_2) = 0.08$, $var(S_3) = 0.18$, $cov(S_1, S_2) = 0.05$, $cov(S_1, S_3) = 0.02$, $cov(S_2, S_3) = 0.03$. Formulate a QPP that can be used to find the portfolio that attains an expected annual return of at least 12% and minimizes the variance of the annual dollar return on the portfolio.*

CorrectAnswer 9. *Let x_j be the number of dollars invested in stock j ($j = 1, 2, 3$). Then the annual return on the portfolio is $(x_1S_1 + x_2S_2 + x_3S_3)/1,000$ and the expected annual return on the portfolio is:*

$$x_1E(S_1) + x_2E(S_2) + x_3E(S_3)$$

To ensure that the portfolio has an expected return of at least 12%, we must include the following constraint in the formulation:

$$\frac{0.14x_1 + 0.11x_2 + 0.10x_3}{1,000} \geq 0.12 \implies 0.14x_1 + 0.11x_2 + 0.10x_3 \geq 120$$

Of course, we must also include the constraint $x_1 + x_2 + x_3 = 1,000$. We assume that the amount invested in a stock must be nonnegative (that is, no short sales of stock are allowed) and add the constraints $x_1, x_2, x_3 \geq 0$. Our objective is simply to minimize the variance of the portfolio's final value:

$$\begin{aligned} var(x_1S_1 + x_2S_2 + x_3S_3) &= var(x_1S_1) + var(x_2S_2) + var(x_3S_3) \\ &\quad + 2cov(x_1S_1, x_2S_2) + 2cov(x_1S_1, x_3S_3) \\ &\quad + 2cov(x_2S_2, x_3S_3) \\ &= x_1^2 varS_1 + x_2^2 varS_2 + x_3^2 varS_3 \\ &\quad + 2x_1x_2\, cov(S_1, S_2) \\ &\quad + 2x_1x_3\, cov(S_1, S_3) + 2x_2x_3\, cov(S_2, S_3) \\ &= 0.20x_1^2 + 0.08x_2^2 + 0.18x_3^2 + 0.10x_1x_2 + 0.04x_1x_3 + 0.06x_2x_3 \end{aligned}$$

Observe that each term in the last expression for the portfolio's variance is of degree 2. Thus, we have an NLP with linear constraints and an objective function consisting of terms of degree 2. To obtain the minimum variance portfolio yielding an expected return of at least 12%, we must solve the following QPP:

$$\begin{aligned} \textit{minimize} \quad & z = 0.20x_1^2 + 0.08x_2^2 + 0.18x_3^2 + 0.10x_1x_2 + 0.04x_1x_3 + 0.06x_2x_3 \\ \textit{subject to} \quad & 0.14x_1 + 0.11x_2 + 0.10x_3 \geq 120 \\ & x_1 + x_2 + x_3 = 1,000 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

```
# Create a new model
m = Model("qp")

# Create variables
x1 = m.addVar(lb=0, name="x1")
x2 = m.addVar(lb=0, name="x2")
x3 = m.addVar(lb=0, name="x3")

# Set objective
m.setObjective(0.20*x1*x1 + 0.08*x2*x2 + 0.18*x3*x3 + 0.10*x1*x2 + 0.04*x1*
    x3 + 0.06*x2*x3, GRB.MINIMIZE)

# Add constraints
m.addConstr(0.14*x1 + 0.11*x2 + 0.10*x3 >= 120, "c0")
m.addConstr(x1 + x2 + x3 == 1000, "c1")

# Optimize model
m.optimize()

# Print solution
print('Optimal Values:')
print('x1:', x1.X)
print('x2:', x2.X)
print('x3:', x3.X)
print('Minimized z:', m.ObjVal)
```